

Costruttori

Costruttori

Come usare i costruttori per inizializzare gli oggetti in C++.

Cos'è un costruttore?

Ogni volta che crei un oggetto, i suoi campi inizialmente contengono valori casuali. Se dimentichi di impostarli prima di usarli, il programma si comporta in modo imprevedibile.

Il **costruttore** risolve questo: è una funzione speciale che viene chiamata **automaticamente** ogni volta che crei un oggetto, e il suo compito è impostare i valori iniziali. Come il modulo di registrazione che compili quando apri un conto in banca.

Come funziona un costruttore

Un costruttore ha tre caratteristiche speciali:

1. Ha lo **stesso nome della classe**
2. **Non ha tipo di ritorno** (nemmeno `void`)
3. Viene chiamato **automaticamente** alla creazione dell'oggetto

```
class Persona {
public:
    string nome;
    int eta;

    // Questo è il costruttore (stesso nome della classe, nessun tipo di
    ritorno)
    Persona() {
        nome = "Sconosciuto"; // imposta valori di default
        eta = 0;
    }
};

int main() {
    Persona p; // il costruttore viene chiamato qui, automaticamente
    cout << p.nome << ", " << p.eta << endl; // Sconosciuto, 0
    return 0;
}
```

Costruttore con parametri

Puoi passare dati al costruttore per inizializzare l'oggetto con valori specifici:

```

class Persona {
public:
    string nome;
    int eta;

    // Costruttore con parametri
    Persona(string n, int e) {
        nome = n;
        eta = e;
    }
};

int main() {
    Persona p1("Alice", 16);    // crea Alice di 16 anni
    Persona p2("Bob", 17);      // crea Bob di 17 anni

    cout << p1.nome << ", " << p1.eta << endl;    // Alice, 16
    cout << p2.nome << ", " << p2.eta << endl;    // Bob, 17
    return 0;
}

```

La lista di inizializzazione

Un modo più efficiente di impostare i campi nel costruttore è la **lista di inizializzazione**, che si scrive dopo i due punti

:

```

class Rettangolo {
private:
    double base;
    double altezza;

public:
    // base(b) significa: inizializza base con il valore di b
    Rettangolo(double b, double a) : base(b), altezza(a) {
        // il corpo può restare vuoto
    }

    double area() const { return base * altezza; }
};

```

È preferita perché inizializza i campi direttamente, senza prima crearli vuoti e poi assegnarli.

Più costruttori per la stessa classe

Puoi avere più costruttori con parametri diversi — il compilatore sceglie quello giusto in base a come crei l'oggetto:

```
class Punto {
public:
    double x;
    double y;

    // Costruttore senza parametri: crea il punto nell'origine
    Punto() : x(0.0), y(0.0) {}

    // Costruttore con un valore: stessa coordinata per x e y
    Punto(double v) : x(v), y(v) {}

    // Costruttore con due valori
    Punto(double x, double y) : x(x), y(y) {}
};

int main() {
    Punto p1;           // (0, 0) – usa il primo costruttore
    Punto p2(5.0);      // (5, 5) – usa il secondo costruttore
    Punto p3(3.0, 4.0); // (3, 4) – usa il terzo costruttore
    return 0;
}
```

Valori di default nei parametri

Puoi dare valori predefiniti ai parametri del costruttore, così non sei obbligato a fornirli tutti:

```

class Cane {
public:
    string nome;
    string razza;
    int eta;

    // Se non specifichi razza o eta, usano i valori di default
    Cane(string n, string r = "Meticcio", int e = 0)
        : nome(n), razza(r), eta(e) {}

    void descrivi() {
        cout << nome << " (" << razza << ", " << eta << " anni)" << endl;
    }
};

int main() {
    Cane d1("Fido", "Labrador", 3);    // tutti i parametri
    Cane d2("Rex", "Pastore");          // eta usa il default (0)
    Cane d3("Luna");                    // razza e eta usano i default

    d1.descrivi(); // Fido (Labrador, 3 anni)
    d2.descrivi(); // Rex (Pastore, 0 anni)
    d3.descrivi(); // Luna (Meticcio, 0 anni)
    return 0;
}

```

Il distruttore: il contrario del costruttore

Come il costruttore crea e inizializza, il **distruttore** fa la pulizia quando l'oggetto non serve più. Si definisce con il simbolo `~` davanti al nome della classe:

```

class Risorsa {
public:
    Risorsa() {
        cout << "Risorsa creata" << endl;
    }

    ~Risorsa() {
        cout << "Risorsa distrutta" << endl;
    }
};

int main() {
    cout << "Inizio" << endl;
    {
        Risorsa r;                // costruttore chiamato qui
        cout << "In uso" << endl;
    }                             // qui r esce dal blocco → distruttore
    chiamato
    cout << "Fine" << endl;
    return 0;
}

```

Output:

```

Inizio
Risorsa creata
In uso
Risorsa distrutta
Fine

```

Il distruttore è utile soprattutto per liberare risorse (memoria, file aperti, connessioni di rete) quando l'oggetto viene eliminato.

Esempio pratico: conto bancario

```
#include <iostream>
#include <string>
using namespace std;

class ContoBancario {
private:
    string intestatario;
    double saldo;

public:
    // Costruttore: apre il conto
    ContoBancario(string nome, double saldoIniziale = 0.0)
        : intestatario(nome), saldo(saldoIniziale) {
        cout << "Conto aperto per " << intestatario << endl;
    }

    // Distruttore: chiude il conto
    ~ContoBancario() {
        cout << "Conto di " << intestatario << " chiuso." << endl;
    }

    void deposita(double importo) { saldo += importo; }
    double getSaldo() const { return saldo; }
    string getNome() const { return intestatario; }
};

int main() {
    ContoBancario c1("Alice", 1000.0); // saldo iniziale 1000
    ContoBancario c2("Bob");           // saldo iniziale 0 (default)

    c1.deposita(500.0);
    cout << c1.getNome() << ": " << c1.getSaldo() << " euro" << endl;
    cout << c2.getNome() << ": " << c2.getSaldo() << " euro" << endl;

    return 0; // qui i distruttori vengono chiamati automaticamente
}
```